Deductive verification of sparse sets in Why3

Catherine Dubois

ENSIIE - Université Paris-Saclay, INRIA, LMF

Introduction

Yet another formally verified implementation of finite sets ...

e.g. formal verification of finite sets based on sorted lists, sequences of ranges, AVL trees, red/black trees in e.g. Isabelle/HOL, Coq or Why3

Here focus on sparse sets, a mutable data structure studied in 1993 by Briggs and Torczon, appearing as an exercice in "The Design and Analysis of Computer Algorithms" (1974, Aho, Hopcroft and Ullman)

Different variants, e.g.:

- sparse arrays, Vacid-0 benchmark (2010)
- sparse sets as domains in constraint solvers (e.g. MiniCP, OsCaR)
- family resemblance to the Knuth's dancing links

Many implementations on the web, in several programming languages
Few formally verified implementations: one about sparse arrays in Why3 (membership/add/remove) done by Filliâtre and Paskevich

Introduction

Yet another formally verified implementation of finite sets ...

e.g. formal verification of finite sets based on sorted lists, sequences of ranges, AVL trees, red/black trees in Isabelle/HOL, Coq or Why3

Here focus on sparse sets, a mutable data structure studied in 1993 by Briggs and Torczon, appearing as an exercice in "The Design and Analysis of Computer Algorithms" (1974, Aho, Hopcroft and Ullman)

Different variants, e.g.:

- sparse arrays, Vacid-0 benchmark (2010)
- **sparse sets as domains** in constraint solvers (e.g. MiniCP, OsCaR)
- family resemblance to the Knuth's dancing links

Many implementations on the web, in several programming languages
Few formally verified implementations: one about sparse arrays in Why3 (membership/add/remove) done by Filliâtre and Paskevich

Context: Sparse sets as domains in constraint solvers

CSP (constraint satisfaction problem) $= (\mathcal{X}, \mathcal{D}, \mathcal{C})$

$\mathcal{X}$: set of variables, $\mathcal{D}(x)$: domain of the x variable, finite set $\mathcal{C}$: set of constraints to be satisfied

Main technics for solving a CSP: domain pruning + enumeration of values + backtracking

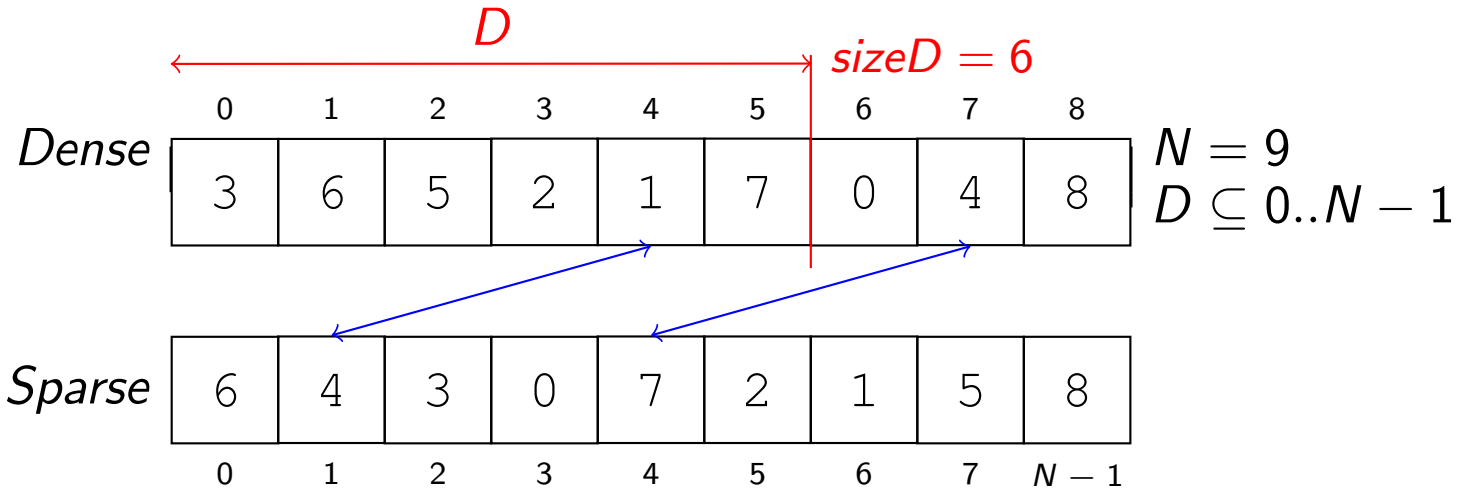
For x a variable, initialize $\mathcal{D}(x) = 0..N-1$, for some N , then propagate constraints to prune $\mathcal{D}(x)$ (remove values), then choose a value for x in $\mathcal{D}(x)$, again propagate, etc., backtrack if needed

Representation of domains: sequences of ranges, bitvectors, sparse sets,...
Sparse sets: efficient operations + easy and cheap when backtracking

$\longrightarrow$ Our main contribution: a formally verified implementation of **sparse sets as reversible/backtrackable domains** developed in Why3 and extracted in OCaml code

Sparse sets as domains - Data structure

2 arrays and an integer to represent $D \subseteq 0..N-1$, $0 \leq N$ (N called the width of the sparse set)



Invariants

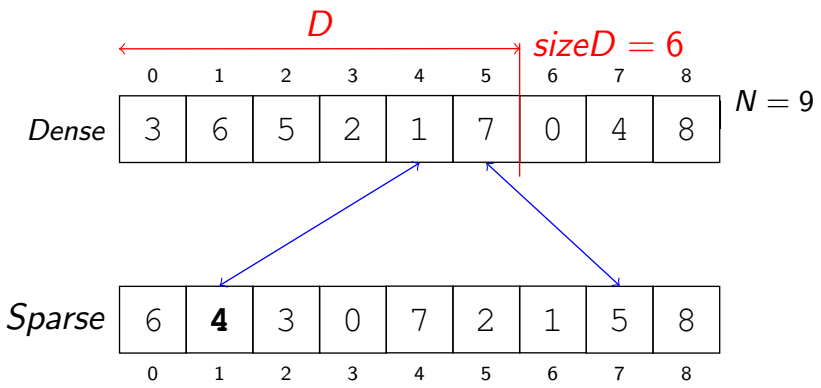
$$D \subseteq 0..N-1 \wedge D = \{Dense[i] \mid 0 \leq i < sizeD\} \quad (P_1)$$

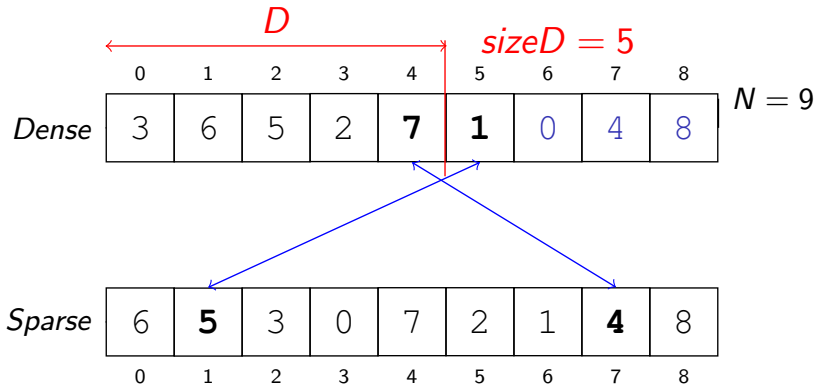
$$Sparse[v] = i \iff Dense[i] = v, \text{ for all } i \text{ and } v \quad (P_2)$$

Sparse sets as domains - Operations

- $v \in D$? $\longrightarrow$ $Sparse[v] < sizeD$? $\longrightarrow$ constant time operation
- remove v from D $\longrightarrow$ exchange v with the last element in *Dense*, decrement *sizeD* and update *Sparse* $\longrightarrow$ constant time operation

  Ex: remove 1





Sparse sets as domains - Operations

- insert v in $D \longrightarrow$ swap v in *Dense* with element at position *sizeD*, update *Sparse*[v] and increment *sizeD* $\longrightarrow$ constant time operation

  clear $\longrightarrow$ $sizeD \leftarrow 0$ $\longrightarrow$ constant time operation

  bind $v \longrightarrow$ swap v in *Dense* with the first element, update *Sparse*[v] and $sizeD \leftarrow 1$ $\longrightarrow$ constant time operation

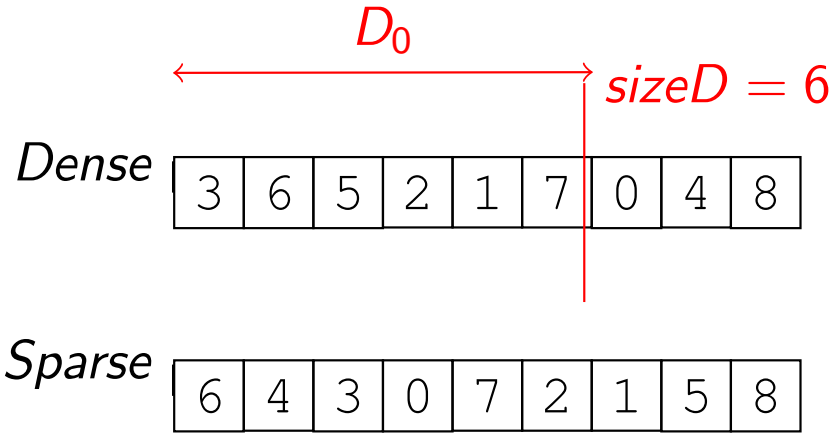
  cardinal $\longrightarrow$ *sizeD* $\longrightarrow$ constant time operation
- exists, forall, tolist, copy $\longrightarrow$ iterate over $Dense[0..sizeD - 1]$ $\longrightarrow$ linear wrt the number of elements in D
- inplace union $A \leftarrow A \cup B \longrightarrow$ iterate over the elements of B (assumed to have the smallest width) $\longrightarrow$ linear wrt the number of elements on B

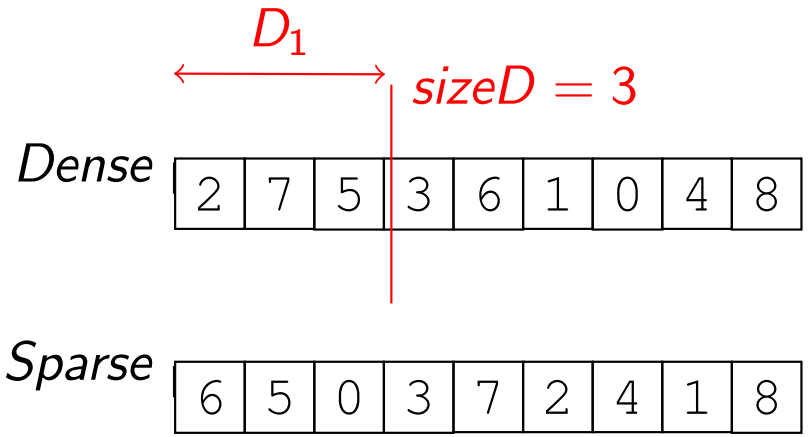
  inplace difference $A \leftarrow A - B \longrightarrow$ iterate over the elements of B $\longrightarrow$ linear wrt the number of elements on B

  inplace intersection $A \leftarrow A \cap B \longrightarrow$ iterate over the elements of A (assumed to have the smallest width) (iteration and removal at the same time) $\longrightarrow$ linear wrt the number of elements on A

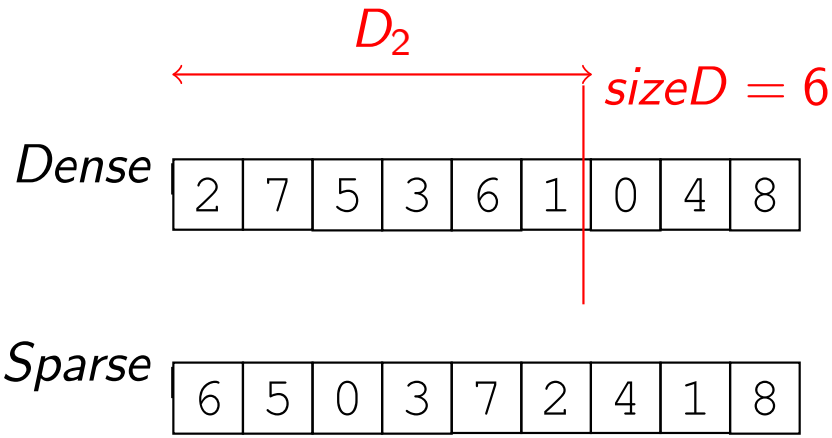
Sparse sets as domains are backtrackable!

The only element to store and restore is *sizeD*





After removing 1, 6 et 3

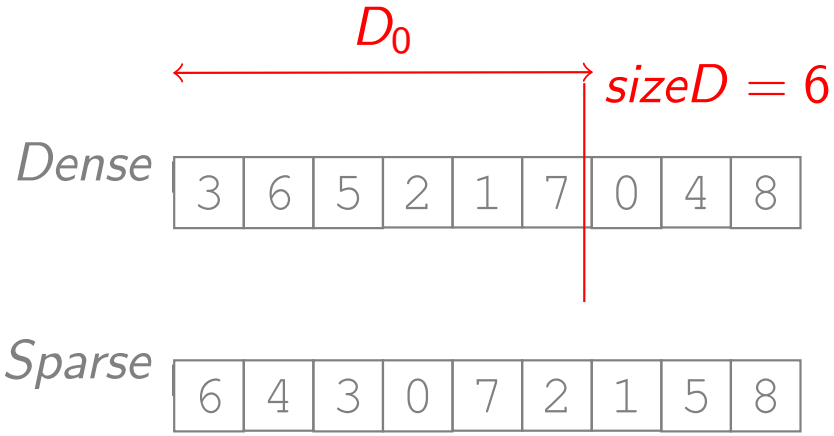


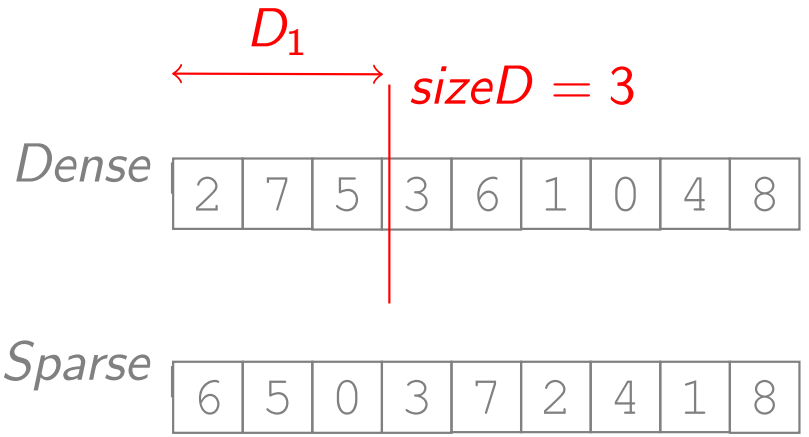
Undo & come back to $D_0 \longrightarrow D_2 = D_1$

Sparse sets as domains are backtrackable!

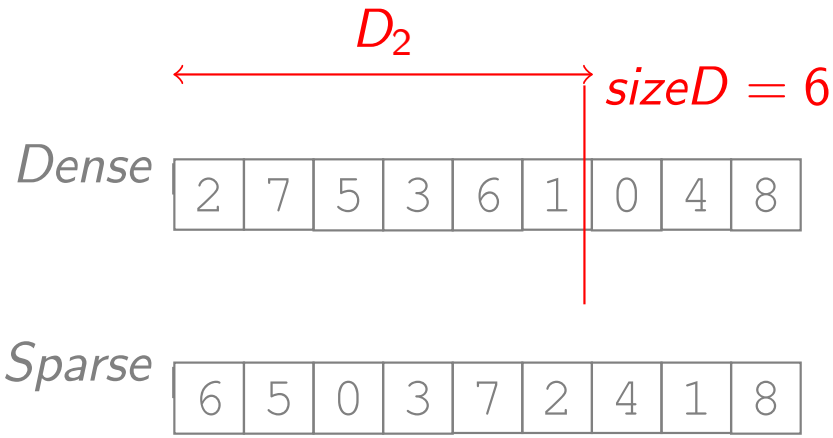
... when no insertion of elements ⚠

The only element to store and restore is *sizeD*





After removing 1, 6 et 3



Undo & come back to $D_0 \longrightarrow D_2 = D_1$

Sparse sets as domains in WhyML & Why3

- Implementation of the data structure and all these operations in WhyML with their specifications
  $\longrightarrow$ A module specifying finite sets + a clone module implementing sparse sets
- Proof of the invariants (Properties P1 and P2) and contracts of the operations

The abstract module

```
module FiniteNatSet

type t = abstract {n : int; mutable setD : fset int;}
invariant {setD ⊆ (interval 0 n)} %P1

val create (nn : int) (c : int) : t
requires {0<=nn}
ensures {result.setD = FsetInt.empty && result.n = nn}

val member (v : int) (a : t) : bool
requires {0<=v}
ensures {result <-> mem v a.setD}

val remove (v : int) (a : t) : unit
requires {0<=v<a.n &&  v ∈ a.setD}   writes {a.setD}
ensures {a.setD == remove v (old a.setD)}

val inter_in_place (a : t) (b : t) : unit
requires {a.n <= b.n}  writes {a.setD}
ensures {a.setD == inter (old a.setD) (b.setD)}
....
```

The concrete implementation

```
type tsparse = { n : int;
         mutable dense: array int;
         mutable sparse: array int;
         mutable sizeD: int;
         mutable ghost setD : fset int;
         }
invariant {
   dom_ran dense n  && dom_ran sparse n  &&
   0 <= sizeD <= n &&
   setD ⊆ (interval 0 n) && %P1
   (forall x:  int.  0<=x<n ->(sparse[x] < sizeD <-> x ∈ setD)) && %P1
   (forall i:int.  0 <= i < sizeD -> sparse[dense[i]]=i)  %P2
           }
```

Operations bind & remove

```
let bind (a : tsparse) ( v : int)
requires 0<=v<a.n
requires {a.sparse[v] < a.sizeD}
requires {a.sizeD > 0}
ensures a.setD == singleton v
=
swap_two_arrays a.dense a.sparse a.n 0 a.sparse[v] ;
a.sizeD <- 1;
a.setD <- singleton v;
```

```
let remove_sparse (v : int) (a : tsparse)
requires 0<=v<a.n && v ∈ a.setD
ensures {same_end a.dense (old a.dense) (old a.sizeD) a.n} %P3
ensures a.setD == remove v (old a.setD)
=
swap_two_arrays a.dense a.sparse a.n a.sparse[v] (a.sizeD - 1);
a.sizeD <- a.sizeD - 1;
a.setD <- remove v a.setD;
```

Intersection - the 'real' code

```
let inter_in_place_sparse (a : tsparse) (b : tsparse)
requires a.n <= b.n
ensures a.setD == inter (old a.setD) (b.setD)
=



let ref i = 0 in
while (i <= a.sizeD - 1) do









  if (b.sparse)[(a.dense)[i]] >= b.sizeD then
          begin
          remove_sparse a.dense[i] a
          end
  else begin                                          i <- i + 1 end
done
```

Intersection - the WhyML code

```
let inter_in_place_sparse (a : tsparse) (b : tsparse)
requires a.n <= b.n
ensures a.setD == inter (old a.setD) (b.setD)
=
let ghost ref visited = FsetInt.empty in
let ghost ref removed = FsetInt.empty in
let ghost ref nb = 0 in
let ref i = 0 in
while (i <= a.sizeD - 1) do
  variant {a.sizeD - i}
  invariant {0<=i && i <= a.sizeD && 0 <= nb && (a.sizeD + nb) <= a.n }
  invariant {dom_ran a.dense a.n}
  invariant {dom_ran a.sparse a.n}
  invariant {comp_is_id a.dense a.sparse a.n}
  invariant {0 <= a.sizeD}
  invariant {sparse_inv1 a}
  invariant {sparse_inv2 a}
  invariant {forall x:int. (exists j. a.sizeD<=j<a.sizeD+nb && x = a.dense[j] &&  x ∉ b.setD)
                                  <->  x ∈ removed}
  invariant {forall x:int. (exists j. 0<=j<i && x = a.dense[j]) <->  x ∈ visited}
  invariant { a.setD == diff (old a.setD) removed}
  invariant {removed == inter (diff (old a.setD) (b.setD)) (union visited removed)}
  if (b.sparse)[(a.dense)[i]] >= b.sizeD then
          begin removed <- add a.dense[i] removed; nb <- nb + 1;
          remove_sparse a.dense[i] a
          end
  else begin   visited <- add a.dense[i] visited; i <- i + 1 end
done
```

Undo Operation

Very simple code ... but how to specify that we come back to the previous situation where the set had `p` elements?

```
let undo_sparse (a : tsparse) (p : int) : unit
requires {cardinal a.setD < p <= a.n}
ensures { a.setD == ... )}
=
 a.sizeD <- p
```

$\longrightarrow$ Maintain the set of all previous states ... as a (abstract and concrete) ghost variable

$\longrightarrow$ Visit again the abstract and concrete modules

Abstract module revisited

In the abstract module, the type `t` becomes:

```
type t  = abstract {n : int ; mutable setD : fset int ;
                    mutable states:  fmap int (fset int);
                   }
invariant {setD ⊆ (interval 0 n) &&   valid_states states setD n
```

```
predicate valid_states (states : fmap int (fset int) (setD : fset int) (n
-- domain of states is included in 0..n
   (dom states) ⊆ (interval |setD| (n+1)) &&
-- current state is registered
   mapsto |setD| setD states &&
-- each registered state contains the current one
   forall i:int, s : fset int. mapsto i s states ->
      s ⊆ (interval 0 n) &&  |setD| = i &&  setD ⊆ s
```

```
val undo (a : t) (p : int) : unit
requires {|a.setD|<p≤a.n}
requires { p ∈ (dom states)}
writes {a.setD, a.states }
ensures {exists s.  mapsto p s (old a).states && a.setD == s}
ensures {forall i.  p < i ≤ a.n ->
          (mapsto i v a.states <-> mapsto i v (old a).states)}
```

\+ update the abstract specification of other operations

```
val remove (v : int) (a : t) : unit
...
ensures {forall i.  0 ≤ i ≤ a.n -> i ≠ |a.setD|-> }
          (mapsto i x a.states <-> mapsto i x (old a).states)}
```

```
val add (v : int) (a : t) : unit
...
ensures { dom a.states = {|a.setD|} })
```

Concrete module revisited

```
 type tsparse =      { n : int;
    mutable dense: array int; mutable sparse: array int;
    mutable sizeD: int; mutable ghost setD: fset int;
    mutable states:  fmap int (fset int);
                     }
invariant {
.... && valid_states states setD n &&
--  the key property that makes the link btw previous and current states
 (forall i:  int, s:  fset int.  0<=i<=n -> mapsto i s states ->
           (forall x.  0<=x<n -> (sparse[x]<i <-> mem x s)))
}
```

\+ simple ghost code in the implementations of the operations to update `a.states`

```
let undo_sparse (a : tsparse) (p : int) : unit
...
=
 let ghost v = a.states p in
 let st = a.states in
  a.setD <- v ; a.states <- ((interval 0 p) << a.states);
   a.sizeD <- p
```

Proof numbers

Statistics per prover: number of proofs, time (minimum/maximum/average) in seconds

The proof of all the VCs are done automatically using two automatic provers, CVC4 and Alt-Ergo using the strategy Auto Level 2.

Prover	*nb.proofs*	*min.time(s)*	*max.time(s)*	*av.time(s)*
Alt-Ergo 2.5.1	37	0.01	1.09	0.29
CVC4 1.6	148	0.05	0.93	0.12

Conclusion and perspectives

Summary:

- Formal verification of sparse sets (as in Briggs et al, and in the particular context of constraints solving)
- Extraction of efficient OCaml code (with native type `int`), extracted code used in a simple sudoku solver (originally written in OCaml by Filliâtre): intense use of backtracking (undo operation)
- Some experimental evaluation/comparison
- Code available (`https://gitlab.com/cdubois/SparseSets`)

Perspectives:

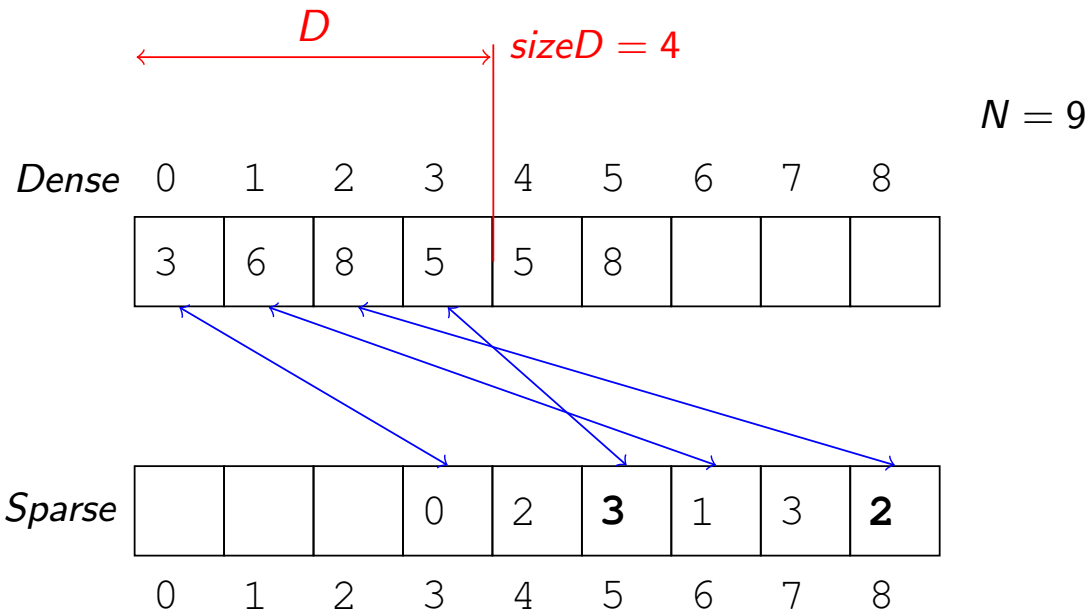
- More advanced experimental evaluation
- Implementation with `sizeD` as a reversible integer
- Integration in CoqBinFD (formally verified solver in Coq)

Thank you for your attention!

Questions?

Sparse sets (Briggs and Torczon) - Data structure

2 arrays and an integer to represent $D \subseteq 0..N-1$, $0 \leq N$ (N called the width of the sparse set)

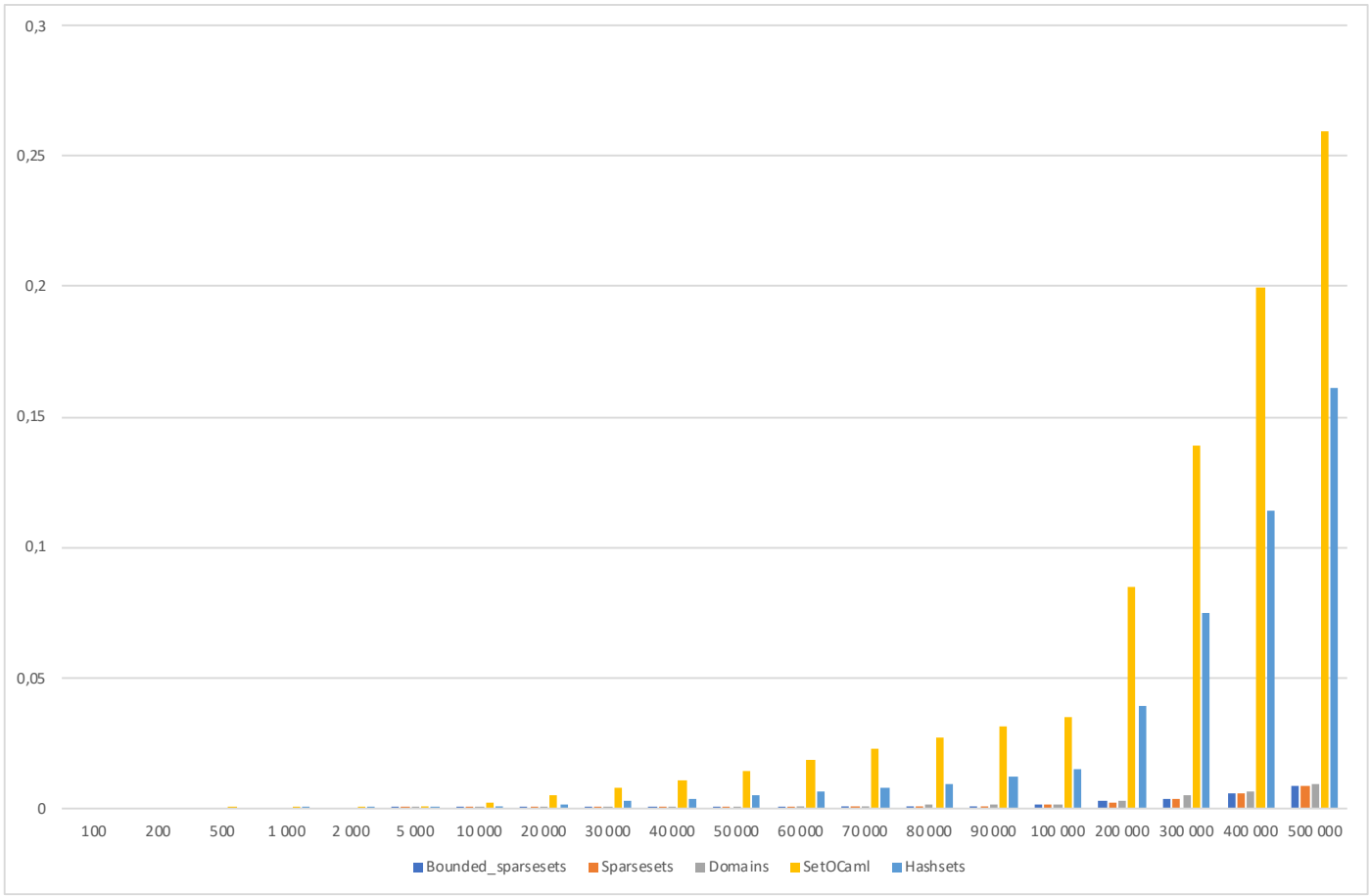


Invariants

$$D \subseteq 0..N-1 \wedge D = \{Dense[i] \mid 0 \leq i < sizeD\} \quad (I_1)$$

$$v \in D \iff Sparse[v] < sizeD \wedge Dense[Sparse[v]] = v \quad (I_2)$$

Performance comparison

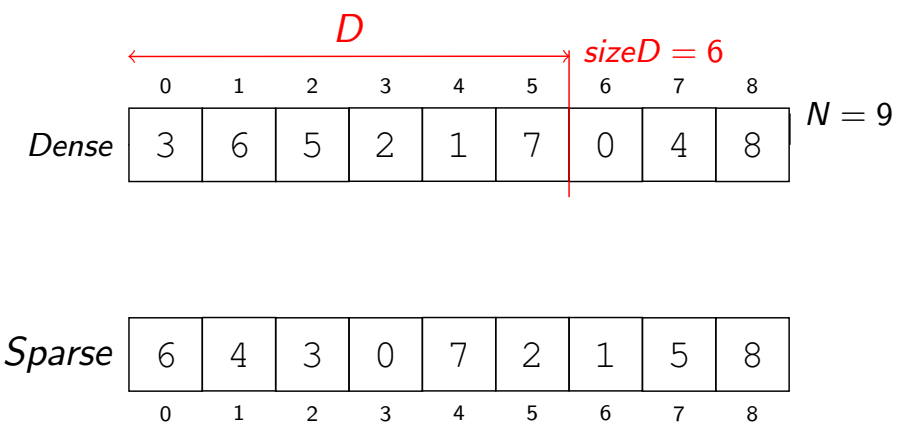


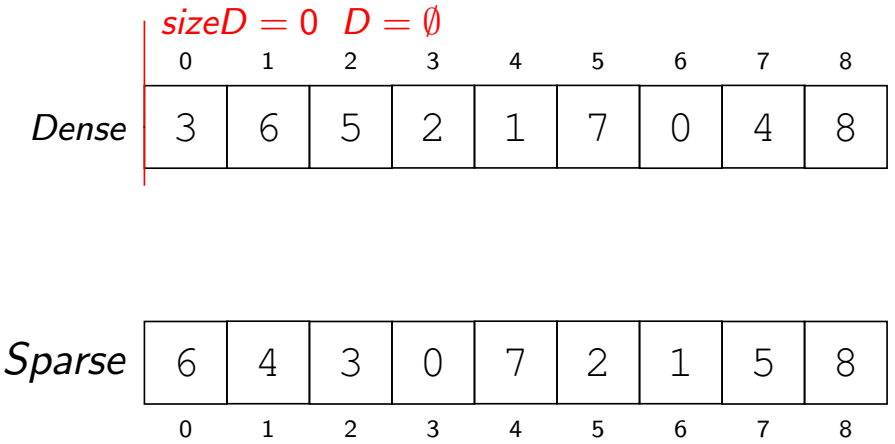
Implementation of a naive implementation of Sieve of Erathosthenes[1] in 3 variants of sparse sets, OCaml std library Set, hashset (J.C. Filliâtre)

[1] www.codeproject.com/Articles/859324/Fast-Implementations-of-Sparse-Sets-in-Cplusplus

Sparse sets - Operations

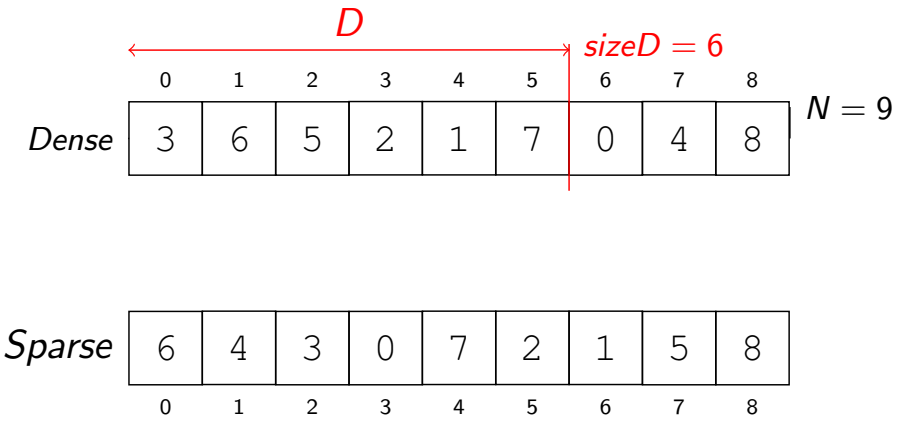
– clear $\longrightarrow$ $sizeD \leftarrow 0$ $\longrightarrow$ constant time operation

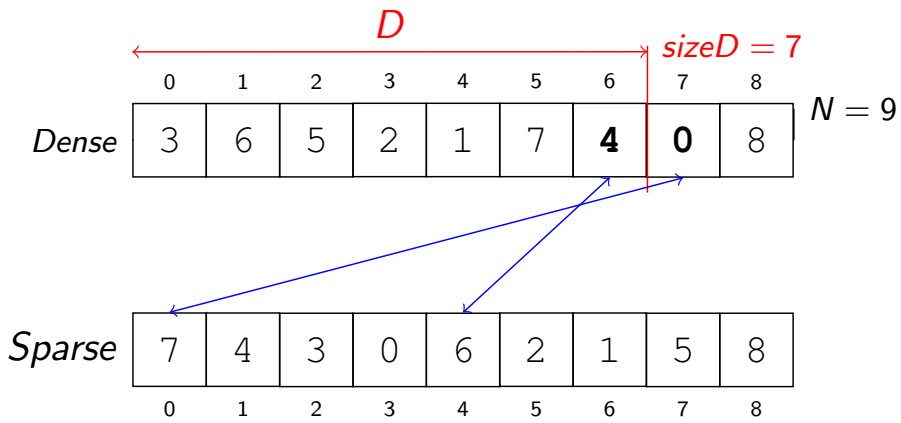




– insert v in D $\longrightarrow$ swap v in *Dense* with element at position *sizeD*, update $Sparse[v]$ and increment *sizeD* $\longrightarrow$ constant time operation

Ex: insert 4





A more 'defensive' implementation

In Why3

```
let test1 () : unit =
let a = full_sparse(8) in
remove_sparse 0 a; remove_sparse 6 a;
remove_sparse 1 a; remove_sparse 4 a;
remove_sparse 2 a;
bind a 5;
undo_sparse a 3;   pre-condition:  proved
assert { mem 3 a.setD && mem 5 a.setD && mem 7 a.setD && not(mem 0 a.setD)}


let test2 () : unit =
let a = full_sparse(8) in
remove_sparse 0 a; remove_sparse 6 a;
remove_sparse 1 a; remove_sparse 4 a;
remove_sparse 2 a;
bind a 5;
assert {a.states[2]= None};
undo_sparse a 2                pre-condition:  proof failed
```

Extracted code for test2: broken invariant!

$\longrightarrow$ Implementation of `a.sizeD` as a reversible int (an int and a stack of its previous values) (work in progress)